Pixel Pattern Classifier

CS212 Final Project

BY HENGMENG LY, JUSTIN KIM AND BELLA HO

Outline

- Introduction and Background
- Motivation and Competitive Analysis Table
- System Architecture
- Sequence Diagram
- Test Cases
- Conclusion

Introduction and Background

Pixel Pattern Classifier is a program that recognizes symbols by analyzing their pixel patterns

The program can:

- Accept a 2D grid of pixels as input
- Process the image to isolate the important symbol
- Convert the symbol into a standard format
- Compare it with known templates
- Output a prediction with confidence

Motivation

Traditional pixel comparison can recognize symbols, but it has limitations:

- Symbols may be different sizes
- Symbols may not be perfectly centered
- Drawings may contain noise or missing pixels
- Slight rotations or skewed lines can reduce accuracy

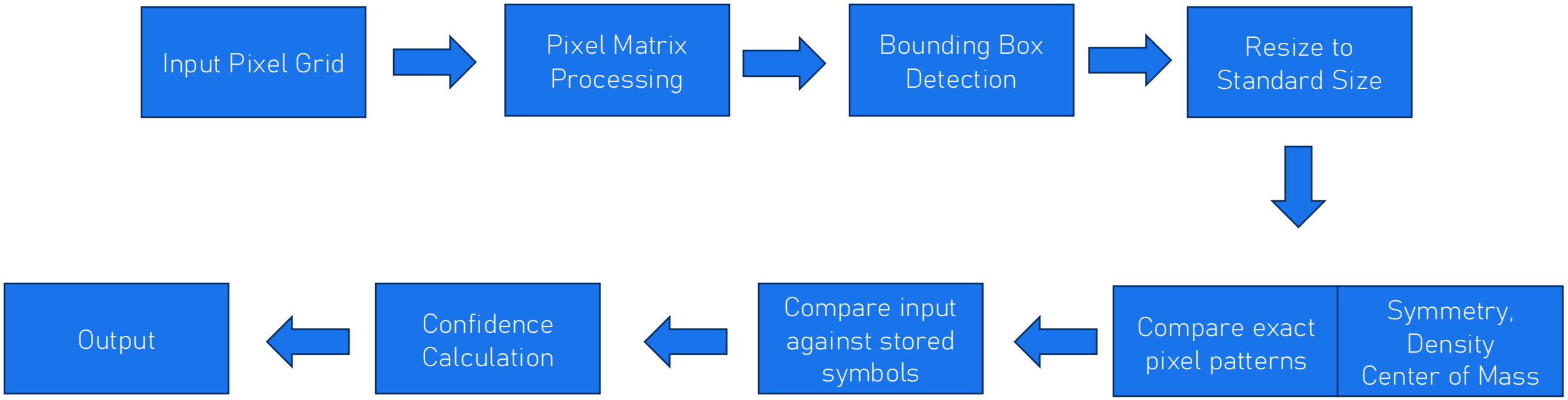
Our goal:

- Create a lightweight symbol recognition system that is understandable, fast, and more resistant to imperfect input.

Competitive Analysis Table

Method	How It Works	Strength	Weakness
Raw pixel matching	Compares each pixels directly	Simple and fast	Fails if pixels shift or noise appears
Template matching	Stores perfect examples and compares inputs	Easy to implement and explain	Needs resizing and alignment
Machine learning	Learns patterns from many examples	Can handle complex images	Requires large datasets and training
Neural Networks	Learns high-level image features	High accuracy	More complex and less interpretable
Pixel Pattern Classifier	Uses templates + structural features	Fast, explainable, handles small variations	Limited to known symbols

System Architecture and Sequence Diagram



Test Cases: 3 Templates

Template 1: X

Template 2: O

Template 3: T

```
int xGrid[max_grid_size][max_grid_size] = {
    {1, 0, 0, 0, 1},
    {0, 1, 0, 1, 0},
    {0, 0, 1, 0, 0},
    {0, 1, 0, 1, 0},
    {1, 0, 0, 0, 1}
};

//O Pattern Grid
int oGrid[max_grid_size][max_grid_size] = {
    {0, 1, 1, 1, 0},
    {1, 0, 0, 0, 1},
    {1, 0, 0, 0, 1},
    {1, 0, 0, 0, 1},
    {0, 1, 1, 1, 0}
};

//T Pattern Grid
int tGrid[max_grid_size][max_grid_size] = {
    {1,1,1,1,1},
    {0,0,1,0,0},
    {0,0,1,0,0},
    {0,0,1,0,0},
    {0,0,1,0,0}
};
```

Test Case: X Pattern

```
Testing clean X pattern:
1 0 0 0 1
0 1 0 1 0
0 0 1 0 0
0 1 0 1 0
1 0 0 0 1

Prediction: X
Confidence: 100%
Runner Up: T
Runner Up Confidence: 47.3949%
Result accepted
```

```
Testing X pattern with noise:
1 0 0 1 1
0 1 0 1 0
0 1 1 0 0
0 1 0 1 0
1 0 0 0 0

Prediction: X
Confidence: 73.0082%
Runner Up: T
Runner Up Confidence: 51.8802%
Result accepted
```

```
Testing heavily noisy X pattern:
1 1 1 0 1
0 1 1 1 0
0 1 1 0 0
0 0 0 1 0
1 0 1 0 0

Prediction: T
Confidence: 63.3324%
Runner Up: X
Runner Up Confidence: 61.7328%
The confidence is too low
```

Test Case: O Pattern

```
Testing clean O pattern:
0 1 1 1 0
1 0 0 0 1
1 0 0 0 1
1 0 0 0 1
0 1 1 1 0

Prediction: O
Confidence: 100%
Runner Up: T
Runner Up Confidence: 47.4835%
Result accepted
```

```
Testing noisy O pattern:
0 1 0 1 0
1 1 0 0 1
1 0 0 0 1
1 1 0 0 1
1 1 1 1 0

Prediction: O
Confidence: 70.693%
Runner Up: X
Runner Up Confidence: 39.6779%
Result accepted
```

Test Case: T Pattern

```
Testing clean T pattern:
1 1 1 1 1
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0
0 0 1 0 0

Prediction: T
Confidence: 100%
Runner Up: O
Runner Up Confidence: 47.4835%
Result accepted
```

```
Testing noisy T pattern:
1 0 1 1 1
0 0 1 0 0
0 0 1 0 1
1 0 1 0 0
0 0 1 0 0

Prediction: T
Confidence: 71.7894%
Runner Up: O
Runner Up Confidence: 49.3176%
Result accepted
```

Test Case: Random and Large X

```
Testing 20x20 X pattern:
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0
0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0
0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0
0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

Prediction: X
Confidence: 55.9652%
Runner Up: T
Runner Up Confidence: 37.1875%
The confidence is too low
```

```
Testing random pattern:
1 1 0 1 0
1 0 0 0 1
0 0 0 0 0
1 0 1 0 1
0 1 0 1 1

Prediction: O
Confidence: 61.9716%
Runner Up: T
Runner Up Confidence: 50.8677%
The confidence is too low
```

Conclusion

The Pixel Pattern Classifier:

✓ Converts pixel grids into usable features

✓ Removes unnecessary background

✓ Resizes symbols for comparison

✓ Uses pixel + structural matching

✓ Predicts symbols with confidence scores

A simple and explainable way to recognize patterns from 2D pixel data.